

Lecture 15: Conceptual Database Design with the ER Model

Amit Kumar Dhar

September 15, 2025

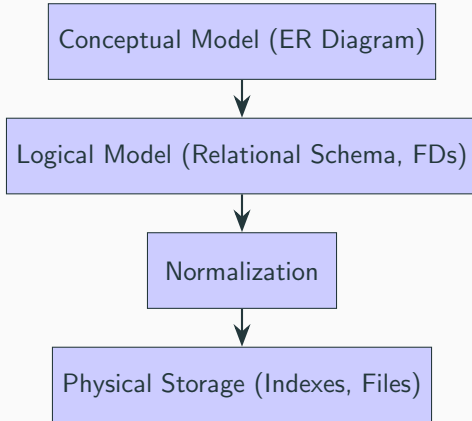
The Database Design Process

Database Design: What, Why, and Why It's Hard

- **What is it?** The process of creating a detailed, formal model of a database. This model defines the entities, their attributes, and the relationships among them.
- **Why is it needed?** To ensure data integrity, avoid data redundancy, and prevent anomalies (insertion, update, deletion). A good design leads to an efficient and reliable database.
- **Why is it hard?** It involves balancing conflicting goals:
 - Data consistency vs. performance.
 - Capturing all requirements vs. keeping the model simple.
 - Flexibility for future changes vs. a stable design.

Layers of Database Design

The design process is typically broken into layers of abstraction.

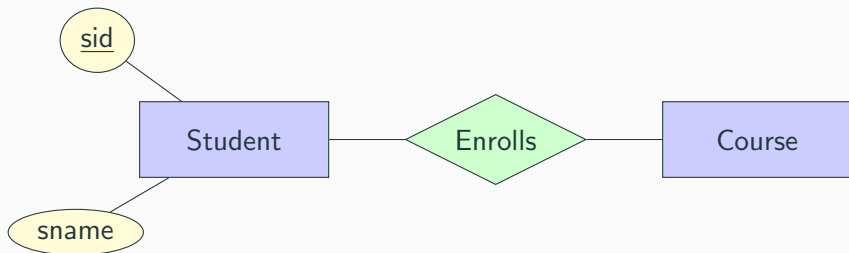


The Entity-Relationship (ER) Model

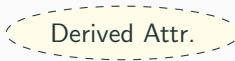
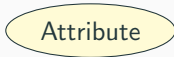
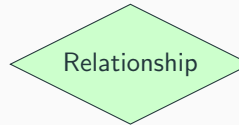
The ER Model: Definition and Example

The Entity-Relationship (ER) model is a high-level, graphical data model used for conceptual database design. It describes the world in terms of:

- **Entities:** Real-world objects (e.g., a student, a course).
- **Attributes:** Properties of entities (e.g., a student's name).
- **Relationships:** Associations among entities (e.g., a student *enrolls* in a course).

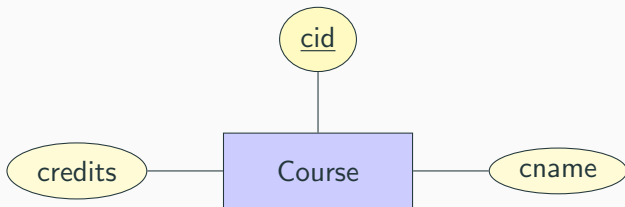


ER Diagram Terminology and Shapes



Keys in ER Diagrams

A key is an attribute (or set of attributes) that uniquely identifies an entity. In ER diagrams, the attributes forming the primary key are **underlined**.

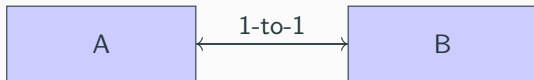


Types of Relationships (Cardinality)

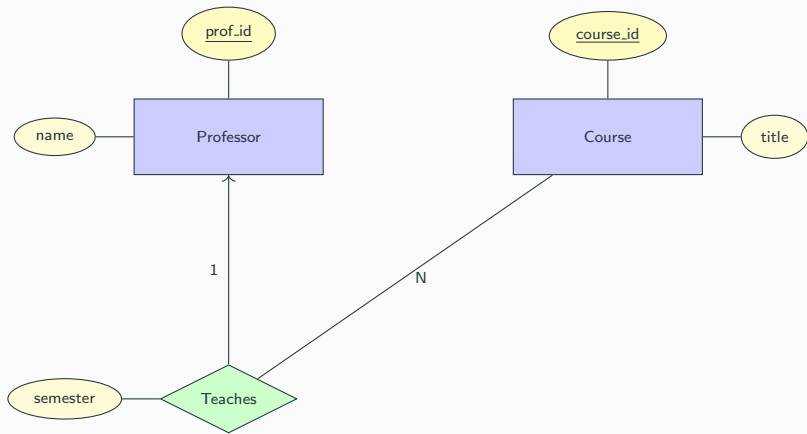
Cardinality defines the number of entities in one set that can be associated with entities in another set.

- **One-to-One:** An entity in A is associated with at most one entity in B. (e.g., A professor *heads* one department).
- **Many-to-One:** Many entities in A can be associated with one entity in B. (e.g., Many students *major in* one department).
- **Many-to-Many:** Many entities in A can be associated with many in B. (e.g., Many students *enroll in* many courses).

Lines can be drawn with arrows (for one) or without (for many).



Complete ER Diagram Example

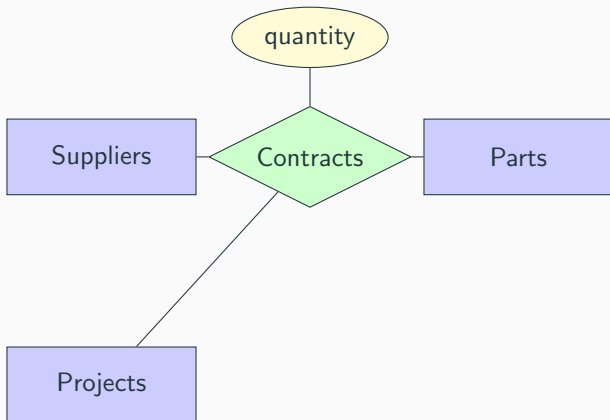


This shows a **many-to-one** relationship: a professor can teach many courses, but a course is taught by one professor. The relationship has an attribute 'semester'.

Multiway Relationships

A relationship that links more than two entity sets.

Example: A 'Contracts' relationship links 'Suppliers', 'Parts', and 'Projects'. A contract specifies that a certain supplier provides a certain part for a certain project.

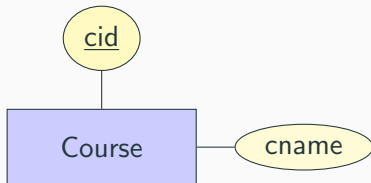


ER to Relational Schema

Converting Entity Sets to Relations

Each entity set is mapped to a new relation (table). The attributes of the entity set become the columns of the table. The key attribute of the entity becomes the primary key of the table.

ER Entity:



Relational Schema:

```
CREATE TABLE Course (  
    cid TEXT PRIMARY KEY,  
    cname TEXT  
);
```

Converting Relationship Sets

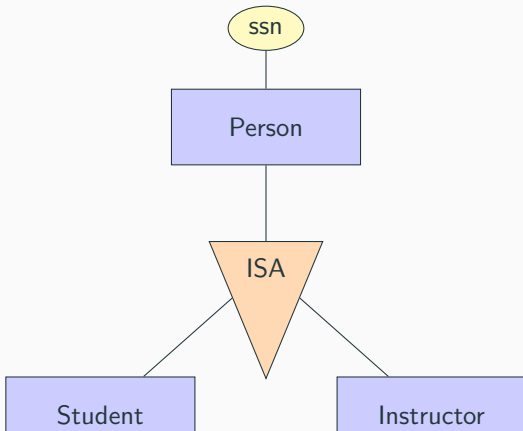
- **Many-to-Many:** Create a new table. The primary key is the combination of the primary keys of the related entities.
 - `Enrolls(Student, Course) → Enrolls(sid, cid, grade)`
- **Many-to-One:** Do not create a new table. Add the primary key of the "one" side to the table of the "many" side as a foreign key.
 - `MajorsIn(Student, Dept) → Add dept_id to the Students table.`
- **Multiway:** Create a new table whose primary key is the combination of the primary keys of all related entities.
 - `Contracts(Suppliers, Parts, Projects) → Contracts(sid, pid, jid, qty)`

Advanced ER Modeling

Modeling Subclasses (ISA Hierarchies)

Subclasses represent specialization within an entity set. An entity in a subclass "is a" member of the superclass and has additional, specific attributes.

Example: 'Instructor' and 'Student' are subclasses of 'Person'.



Converting Subclasses to Relations

There are three main strategies:

1. **ER Style:** A table for each entity set.

```
CREATE TABLE Person(ssn, name);  
CREATE TABLE Student(ssn, gpa); -- ssn is PK and FK  
CREATE TABLE Instructor(ssn, salary); -- ssn is PK and
```

2. **Object-Oriented Style:** One table per subclass, including inherited attributes.

```
CREATE TABLE Student(ssn, name, gpa);  
CREATE TABLE Instructor(ssn, name, salary);
```

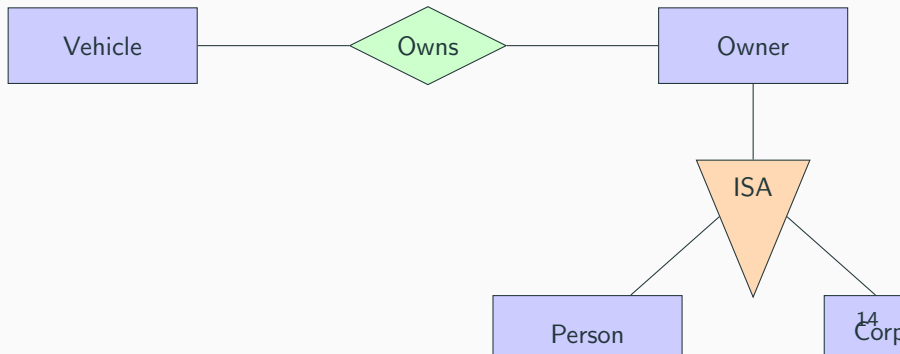
3. **Null-Value Style:** One table for everything.

```
CREATE TABLE Person(ssn, name, gpa, salary, type);
```

Modeling Union Types with Subclasses

A union type occurs when a relationship can be with one of several distinct entity sets. This is best modeled by creating a common superclass.

Example: A 'Vehicle' is owned by either a 'Person' or a 'Corporation'.



Thank You!
Questions?